GUIA TÉCNICO: ELASTIC STACK AVANÇADO

Filebeat Autodiscover • Index Lifecycle Management (ILM) • Logstash Optimization

Engenharia de Observabilidade & DevOps | Guia de Produção

1. Filebeat Autodiscover Avançado

O recurso de **Autodiscover** do Filebeat permite monitorar infraestruturas dinâmicas em containers (Docker e Kubernetes). Ele escuta eventos do daemon do Docker ou da API do Kubernetes para criar e encerrar dinamicamente as configurações de coleta de logs.

1.1 Templates vs. Hints-based Autodiscover

> **Diferença Chave:** O autodiscover baseado em *Templates* centraliza todas as regras no arquivo de configuração do Filebeat. Já o modelo baseado em *Hints* permite que os próprios desenvolvedores definam o comportamento de coleta através de etiquetas (labels/annotations) diretamente no manifesto do container.

1.2 Configuração Dinâmica com Docker Labels (Hints)

Abaixo está o exemplo de um manifesto Docker Compose configurando hints avançados para parsing JSON e multiline:

```
labels:
  co.elastic.logs/enabled: "true"
  co.elastic.logs/json.keys_under_root: "true"
  co.elastic.logs/json.add_error_key: "true"
  co.elastic.logs/pipeline: "microservices-pipeline"
  co.elastic.logs/multiline.pattern: '^\['
  co.elastic.logs/multiline.negate: "true"
  co.elastic.logs/multiline.match: "after"
```

1.3 Appenders Avançados

Os *Appenders* permitem injetar configurações condicionais adicionais no momento em que o container é descoberto pelo Filebeat:

```
filebeat.autodiscover:
  providers:
    - type: docker
      hints.enabled: true
      appenders:
        - type: config
          condition.contains:
            docker.container.image: "nginx"
          config:
            fields:
              service_type: "webserver"
            fields_under_root: true
```

GUIA TÉCNICO: ELASTIC STACK AVANÇADO

Filebeat Autodiscover • Index Lifecycle Management (ILM) • Logstash Optimization

Engenharia de Observabilidade & DevOps | Guia de Produção

2. Retenção e Ciclo de Vida de Índices (ILM) no Elasticsearch

O **Index Lifecycle Management (ILM)** automatiza a transição de dados entre diferentes níveis de armazenamento (Data Tiers), reduzindo drasticamente os custos operacionais com hardware enquanto mantém a alta performance de busca nos dados recentes.

Data Tier	Perfil de Hardware	Ações ILM Principais	Frequência de Uso
Hot	SSDs de Alta Performance (NVMe)	Rollover por tamanho (50GB) ou idade (30d)	Escrita ativa & busca intensa
Warm	SSDs Padrão / HDDs Rápidos	ForceMerge (1 segment), Shrink shards	Apenas leitura / consultas ocasionais
Cold	HDDs de Baixo Custo / Object Store	Searchable Snapshots, Redução de réplicas	Consultas raras / auditoria
Delete	N/A	Deleção definitiva do índice	Expiração da retenção (ex: 90 dias)

2.1 Exemplo de Política ILM via API REST

```
PUT _ilm/policy/microservices-logs-policy
{
  "policy": {
    "phases": {
      "hot": {
        "min_age": "0ms",
        "actions": {
          "rollover": { "max_primary_shard_size": "50gb", "max_age": "30d" }
        }
      },
      "warm": {
        "min_age": "7d",
        "actions": { "forcemerge": { "max_num_segments": 1 } }
      },
      "cold": {
        "min_age": "30d",
        "actions": { "set_priority": { "priority": 0 } }
      },
      "delete": {
        "min_age": "90d",
        "actions": { "delete": {} }
      }
    }
  }
}
```

3. Melhores Práticas para Otimizar o Logstash

O Logstash executa filtros e saídas em threads paralelas. Para atingir a máxima eficiência de processamento sem sobrecarregar a JVM, deve-se aplicar o tuning correto de workers, memória heap e estrutura de filtros.

3.1 Parâmetros do `logstash.yml`

• **pipeline.workers**: Ajuste para o número de núcleos físicos de CPU. Se o pipeline realizar muitas chamadas I/O de rede, aumente para até 1.5x vCPUs.

• **pipeline.batch.size**: Aumente de 125 para **250 - 1000** em ambientes de alto débito. Lotes maiores otimizam o envio de chamadas Bulk ao Elasticsearch.

• **queue.type: persisted**: Ative para criar uma fila persistente em disco (PQ), absorvendo picos de tráfego sem perder mensagens em caso de queda do container.

3.2 Dimensionamento de Heap JVM (`jvm.options`)

> **Regra dos 50%:** Nunca aloque mais de 50% da memória RAM total do servidor para o Logstash (-Xms e -Xmx idênticos). Deixe o restante da RAM para o cache de sistema operacional. Mantenha o heap sempre abaixo de 31GB para preservar o recurso de Compressed OOPs.

3.3 Otimização do Filtro: Dissect vs. Grok

Sempre que os logs possuírem delimitadores fixos (como logs de acesso Nginx/Apache), substitua o filtro **grok** pelo **dissect**. O *dissect* não utiliza expressões regulares, sendo até **10 vezes mais rápido** e consumindo significativamente menos CPU.

```
filter {
  dissect {
    mapping => { "message" => "%{client_ip} %{ident} %{auth} [%{timestamp}] \"%{verb} %{request} HTTP
/%{httpversion}\" %{status} %{bytes}" }
  }
}
```